Exceptional Handling

In any programming language there are 2 types of errors are possible.

1) Syntax Errors

2) Runtime Errors

1) Syntax Errors:

The errors which occur because of invalid syntax are called syntax errors.

Ex:-

```
x = 10
if x == 10
print("Hello") →SyntaxError: invalid syntax
```

Note: Programmer is responsible to correct these syntax errors. Once all syntax errors are corrected then only program execution will be started.

2) Runtime Errors:

- Also known as exceptions.
- While executing the program if something goes wrong because of end user input or programming logic or memory problems etc. then we will get Runtime Errors.

Ex:-

- print(10/0) → ZeroDivisionError: division by zero
- print(10/"ten") → TypeError: unsupported operand type(s) for /: 'int' and 'str'

What is Exception?

An unwanted and unexpected event that disturbs normal flow of program is called exception.

Eg:

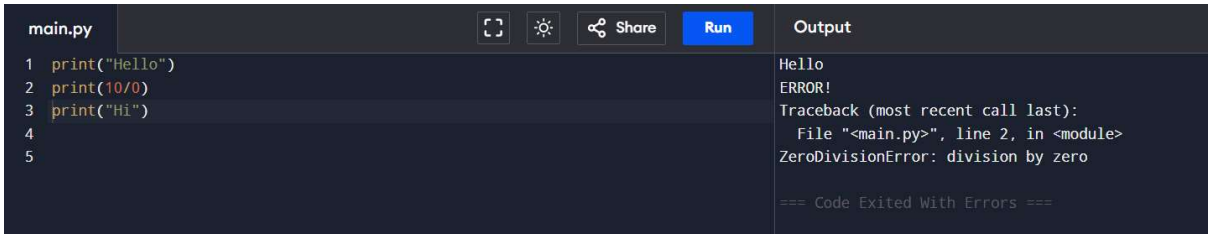
- **ZeroDivisionError**
- **TypeError**
- **ValueError**
- **FileNotFoundError**
- **EOFError**
- **SleepingError**
- **TyrePuncturedError**

It is highly recommended to handle exceptions. The main objective of exception handling is Graceful Termination of the program (i.e we should not block our resources and we should not miss anything)

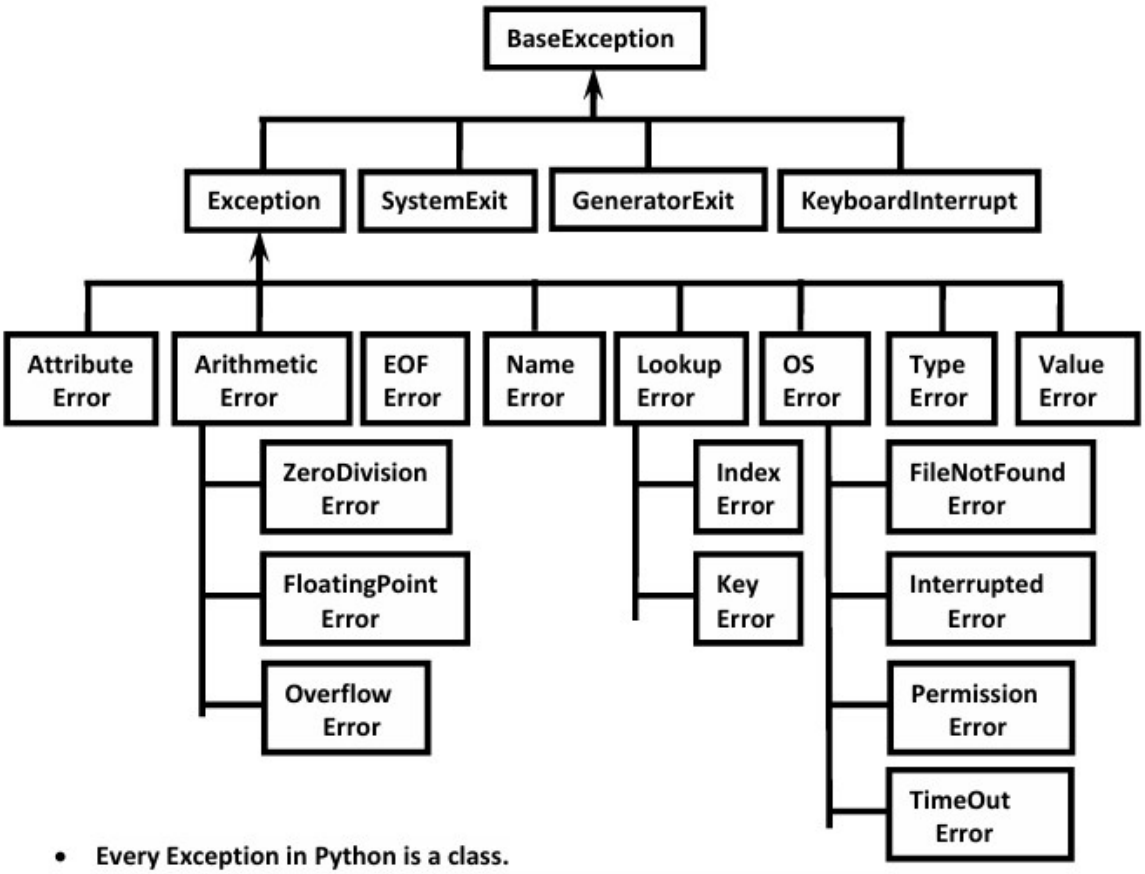
Exception handling does not mean repairing exception. We have to define alternative way to continue rest of the program normally.

Default Exception Handing in Python:

- Every exception in Python is an object. For every exception type the corresponding classes are available.
- Whenever an exception occurs PVM will create the corresponding exception object and will check for handling code. If handling code is not available, then Python interpreter terminates the program abnormally and prints corresponding exception information to the console.
- The rest of the program won't be executed.



Python's Exception Hierarchy



- Every Exception in Python is a class.
- All exception classes are child classes of BaseException.i.e every exception class extends BaseException either directly or indirectly. Hence BaseException acts as root for Python Exception Hierarchy.
- Most of the times being a programmer we have to concentrate Exception and its child classes.

Customized Exception Handling by using try-except:

- It is highly recommended to handle exceptions.
- The code which may raise exception is called risky code and we have to take risky code inside try block. The corresponding handling code we have to take inside except block.

```
try:
    Risky Code
except XXX:
    Handling code/Alternative Code
```

Without try-except:

```
1) print("stmt-1")
2) print(10/0)
3) print("stmt-3")
```

Output

stmt-1
ZeroDivisionError: division by zero
Abnormal termination/Non-Graceful Termination

With try-except:

```
1) print("stmt-1")
2) try:
3)     print(10/0)
4) except ZeroDivisionError:
5)     print(10/2)
6) print("stmt-3")
```

Output

stmt-1
5.0
stmt-3
Normal termination/Graceful Termination

Conclusions:

1) Within the try block if anywhere exception raised then rest of the try block won't be executed eventhough we handled that exception. Hence we have to take only risky code inside try block and length of the try block should be as less as possible.
2) In addition to try block, there may be a chance of raising exceptions inside except and finally blocks also.
3) If any statement which is not part of try block raises an exception then it is always abnormal termination.

How to Print Exception Information:

try:

```
1) print(10/0)
2) except ZeroDivisionError as msg:
3)     print("exception raised and its description is:",msg)
```

Output exception raised and its description is: division by zero

```
1) try:
2)     x=int(input("Enter First Number: "))
3)     y=int(input("Enter Second Number: "))
4)     print(x/y)
5) except ZeroDivisionError :
6)     print("Can't Divide with Zero")
7) except ValueError:
8)     print("please provide int value only")
```

```
D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: 2
5.0

D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: 0
Can't Divide with Zero

D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: ten
please provide int value only
```

Single except Block that can handle Multiple Exceptions:

We can write a single except block that can handle multiple different types of exceptions.

```
except (Exception1,Exception2,exception3,..):    OR
except (Exception1,Exception2,exception3,..) as msg :
```

Parentheses are mandatory and this group of exceptions internally considered as tuple.

```
1) try:
2)     x=int(input("Enter First Number: "))
3)     y=int(input("Enter Second Number: "))
4)     print(x/y)
5) except (ZeroDivisionError,ValueError) as msg:
6)     print("Plz Provide valid numbers only and problem is: ",msg)
```

```
D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: 0
Plz Provide valid numbers only and problem is:  division by zero

D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: ten
Plz Provide valid numbers only and problem is:  invalid literal for int() with b are 10: 'ten'
```

Default except Block:

We can use default except block to handle any type of exceptions.
In default except block generally we can print normal error messages.

Syntax:

```
except:
    statements
```

```
1) try:
2)     x=int(input("Enter First Number: "))
3)     y=int(input("Enter Second Number: "))
4)     print(x/y)
5) except ZeroDivisionError:
6)     print("ZeroDivisionError:Can't divide with zero")
7) except:
8)     print("Default Except:Plz provide valid input only")
```

```
D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: 0
ZeroDivisionError:Can't divide with zero

D:\Python_classes>py test.py
Enter First Number: 10
Enter Second Number: ten
Default Except:Plz provide valid input only
```

***Note:** If try with multiple except blocks available then default except block should be last, otherwise we will get SyntaxError.

```
1) try:
2)     print(10/0)
3) except:
4)     print("Default Except")
5) except ZeroDivisionError:
6)     print("ZeroDivisionError")
```

```
SyntaxError: default 'except:' must be last
```

finally Block:

It is not recommended to maintain clean up code (Resource Deallocating Code or Resource Releasing code) inside try block because there is no guarantee for the execution of every statement inside try block always.

It is not recommended to maintain clean up code inside except block, because if there is no exception then except block won't be executed.

Hence we required some place to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether exception handled or not handled. Such type of best place is nothing but finally block.

Hence the main purpose of finally block is to maintain clean up code.

Case-1: If there is no exception

```
1) try:
2)     print("try")
3) except:
4)     print("except")
5) finally:
6)     print("finally")
```

Output

```
try
finally
```

Case-2: If there is an exception raised but handled

```
1) try:
2)     print("try")
3)     print(10/0)
4) except ZeroDivisionError:
5)     print("except")
6) finally:
7)     print("finally")
```

Output

```
try
except
finally
```

Case-3: If there is an exception raised but not handled

```
1) try:
2)     print("try")
3)     print(10/0)
4) except NameError:
5)     print("except")
6) finally:
7)     print("finally")
```

Output

```
try
finally
ZeroDivisionError: division by zero(Abnormal Termination)
```

else Block with try-except-finally:

We can use else block with try-except-finally blocks.
else block will be executed if and only if there are no exceptions inside try block.

```
try:
        Risky Code
except:
        will be executed if exception inside try


else:
        will be executed if there is no exception inside try
finally:
        will be executed whether exception raised or not raised and handled or not
handled
```

Eg:

```
try:
        print("try")
        print(10/0) → 1
except:
        print("except")
else:
        print("else")
finally:
        print("finally")
```

Various possible Combinations of try-except-else-finally:

1) Whenever we are writing try block, compulsory we should write except or finally block i.e. without except or finally block we cannot write try block.

2) Whenever we are writing except block, compulsory we should write try block. i.e. except without try is always invalid.

3) Whenever we are writing finally block, compulsory we should write try block. i.e. finally without try is always invalid.

4) We can write multiple except blocks for the same try, but we cannot write multiple finally blocks for the same try

5) Whenever we are writing else block compulsory except block should be there. i.e without except we cannot write else block.

6) In try-except-else-finally order is important.

7) We can define try-except-else-finally inside try, except, else and finally blocks. i.e nesting of try-except-else-finally is always possible.

Example:

1	try: print("try")	✗
2	except: print("Hello")	✗
3	else: print("Hello")	✗
4	finally: print("Hello")	✗

5	try: print("try") except: print("except")	✓
6	try: print("try") finally: print("finally")	✓

7	try:
	print("try")
	except:
	print("except")
	else:
	print("else")
		✓

8	try:
	print("try")
	else:
	print("else")
		✗

9	try:
	print("try")
	else:
	print("else")
	finally:
	print("finally")
		✗

10	try: print("try") except XXX: print("except-1") except YYY: print("except-2")	✓
11	try: print("try") except : print("except-1")	✗

else: print("else") else: print("else")

12	try: print("try") except : print("except-1") finally: print("finally") finally: print("finally")	✗
13	try: print("try") print("Hello") except: print("except")	✗
14	try: print("try") except: print("except") print("Hello") except: print("except")	✗
15	try: print("try") except: print("except") print("Hello") finally: print("finally")	✗

16	try:
	print("try")
	except:
	print("except")
	print("Hello")
	else:
	print("else")
		✗

	except: print("except")	
18	try: print("try") except: print("except") try: print("try") finally: print("finally")	✓

19	try:
	print("try")
	except:
	print("except")
	if 10>20:
	print("if")
	else:
	print("else")
		✓

20	try: print("try") try: print("inner try") except: print("inner except block") finally: print("inner finally block") except: print("except")	✓
21	try: print("try") except: print("except") try: print("inner try") except: print("inner except block") finally: print("inner finally block")	✓

23	try:
	print("try")
	except:
	print("except")
	try:
	print("try")
	else:
	print("else")
		✗

24	try: print("try") try: print("inner try") except: print("except")	✗

25	try:
	print("try")
	else:
	print("else")
	except:
	print("except")
	finally:
	print("finally")
		✗

Types of Exceptions:

In Python there are 2 types of exceptions are possible.

1) Predefined Exceptions

2) User Defined Exceptions

1) Predefined Exceptions:

- Also known as inbuilt exceptions.
- The exceptions which are raised automatically by Python virtual machine whenever a particular event occurs are called pre-defined exceptions.

Eg 1: Whenever we are trying to perform Division by zero, automatically Python will raise ZeroDivisionError.

```
print(10/0)
```

2) User Defined Exceptions:

- Also known as Customized Exceptions or Programmatic Exceptions

- Some time we have to define and raise exceptions explicitly to indicate that something goes wrong, such type of exceptions is called User Defined Exceptions or Customized Exceptions

- Programmer is responsible to define these exceptions and Python not having any idea about these. Hence we have to raise explicitly based on our requirement by using "raise" keyword.

Eg:-

- InSufficientFundsException
- InvalidInputException
- TooYoungException
- TooOldException

How to Define and Raise Customized Exceptions:

Every exception in Python is a class that extends Exception class either directly or indirectly.

Syntax:

```
class classname(predefined exception class name):
    def __init__(self,arg):
        self.msg=arg
```

```
1) class TooYoungException(Exception):
2)     def __init__(self,arg):
3)         self.msg=arg
```

TooYoungException is our class name which is the child class of Exception

Program:

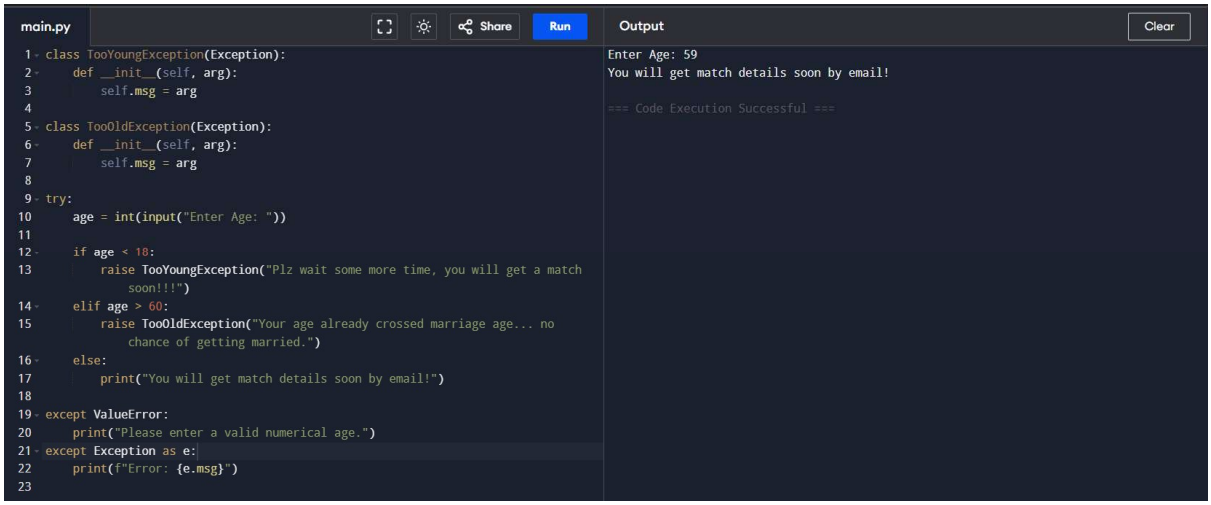
```
class TooYoungException(Exception):
   def __init__(self, arg):
      self.msg = arg

class TooOldException(Exception):
   def __init__(self, arg):
      self.msg = arg

try:
   age = int(input("Enter Age: "))

   if age < 18:
      raise TooYoungException("Plz wait some more time, you will get a match soon!!!")
   elif age > 60:
      raise TooOldException("Your age already crossed marriage age... no chance of getting
married.")
   else:
      print("You will get match details soon by email!")

except ValueError:
   print("Please enter a valid numerical age.")
except Exception as e:
   print(f"Error: {e.msg}")
```



Python Exception Handling quiz

Question 1

What is an exception in Python?

- (A) A syntax error
- (B) A runtime error
- (C) A logical error
- (D) A compile-time error

Question 2

How can you handle exceptions in Python?

- (A) Using if-else statements
- (B) Using try-except blocks
- (C) Using switch-case statements
- (D) Using for loops

Question 3

What is the purpose of the finally block in exception handling?

(A) To define a block of code that will be executed if an exception occurs

(B) To specify the code to be executed regardless of whether an exception occurs or not

(C) To catch and handle specific exceptions

(D) To raise a custom exception

Question 4

Which of the following statements is used to raise a custom exception in Python?

(A) throw

(B) raise

(C) catch

(D) except

Question 5

What will be the output of the following code?

```
try:
    res = 10 / 0
except ZeroDivisionError:
    res = "Infinity"
print(res)
```

(A) 10

(B) "Infinity"

(C) ZeroDivisionError

(D) None

Question 6

What is the purpose of the else block in exception handling?

(A) To handle exceptions

(B) To specify code that will be executed if no exceptions are raised

(C) To define custom exceptions

(D) To skip the execution of the block if an exception occurs

Question 7

How can you catch multiple exceptions in a single except block?

(A) Using a comma-separated list of exceptions

(B) Using nested try-except blocks

(C) By defining a custom exception

(D) Using the catch keyword

Question 8

What is the purpose of the assert statement in Python exception handling?

(A) To catch and handle exceptions

(B) To raise a custom exception

(C) To check if a given expression is true, otherwise raise an 'AssertionError'

(D) To ignore exceptions

Question 9

Will the finally block execute if an exception occurs and is not handled by any except block?

(A) It doesn't execute

(B) It executes before the exception is raised

(C) It executes after try/except blocks

(D) It executes only if the exception is caught

Question 10

What is the purpose of the with statement in Python exception handling?

(A) To create a new exception

(B) To simplify resource management using context managers

(C) To catch and handle exceptions

(D) To define a custom exception

Question 11

How can you handle uncaught exceptions globally in Python?

(A) By defining a custom global exception handler function and assigning it to sys.excepthook

(B) Using the 'global_exception_handler()' function

(C) By using the 'global' keyword

(D) There is no way to handle exceptions globally

Question 12

In Python, what does the sys.exc_info() function return?

(A) The exception type

(B) The exception value

(C) A tuple containing the exception type, value, and traceback

(D) A dictionary containing exception information

Question 13

What is the purpose of the raise statement in Python?

(A) To raise an arbitrary exception

(B) To terminate the program

(C) To catch an exception

(D) To ignore an exception

Question 14

What will happen if an exception is raised but not caught in a Python program?

(A) The program will terminate abruptly

(B) The program will continue to execute without any impact

(C) The program will print an error message and continue

(D) The program will prompt the user to handle the exception

Question 15

How can you re-raise an exception in Python?

(A) Using the retry statement

(B) Using the throw statement

(C) Using the raise statement without any arguments

(D) Using the rethrow keyword

Question 16

Which of the following is the correct syntax to catch an exception and store its details in a variable?

(A) except Exception e:

(B) except Exception, e:

(C) catch Exception as e:

(D) except Exception as e

ANSWERS:

1 → B
2 → B
3 → B
4 → B
5 → B
6 → B
7 → A
8 → C
9 → C
10 → B

11 → B
12 → C
13 → A
14 → A
15 → C
16 → D

Questions were from
https://www.geeksforgeeks.org/quizzes/python-exception-handling-quiz/

What is the difference between an error and an exception?
Errors are issues in code execution and can be categorized into syntax errors (parsing errors) and exceptions (runtime errors). Exceptions are a broader class that also includes logical errors that arise during execution.

- **Should you use a bare `except` statement (e.g., `except:`)?**

  No, it is better to catch specific exceptions to avoid hiding unexpected bugs, as recommended in.